
Disciplina: DCA 3703 - PROGRAMAÇÃO PARALELA

Professor: Samuel Xavier De Souza

Nome: José Felix Rodrigues Anselmo

Tarefa 12 - Avaliação de Desempenho e Escalabilidade Paralela

1. Enunciado

Avalie a escalabilidade do seu código de Navier-Stokes utilizando algum nó de computação do NPAD. Procure identificar gargalos de escalabilidade e reporte o seu progresso em versões sucessivas da evolução do código otimizado. Comente sobre a escalabilidade, a escalabilidade fraca e a escalabilidade forte das versões.

2. Metodologia

Como o enunciado requer um pouco do entendimento sobre a equação de Navier-Stokes para que seja possível entender a execução do código em C.

2.1. O que é a equação de Navier-Stokes

As equações de Navier-Stokes formam o conjunto fundamental de equações diferenciais parciais não lineares que descrevem o movimento de fluidos (líquidos e gases). Elas aplicam as leis de Newton da mecânica (especificamente a conservação do momento linear e da massa) a meios contínuos.

Componentes Principais da Equação de Momento: para um fluido incompressível com densidade constante, a formulação geral para o vetor velocidade $\mathbf{u}$ é expressa como:

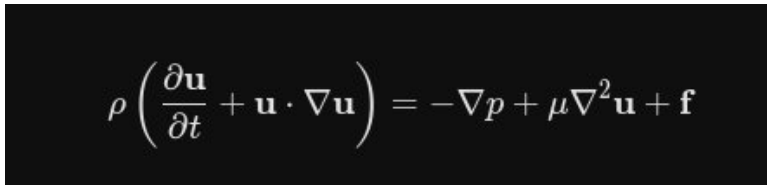

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}$$

Figura 1 — Equação do fluido incompressível

Cada termo representa uma força física atuando sobre o fluido:

- $\partial \mathbf{u} / \partial t$: Aceleração local (como a velocidade muda com o tempo em um ponto fixo).

- $\rho(\mathbf{u} \cdot \nabla \mathbf{u})$: Advecção (ou convecção), que representa o transporte de momento pelo próprio fluxo do fluido (termo não linear responsável pela complexidade e turbulência).
- $-\nabla p$: Gradiente de pressão, as forças geradas por diferenças de pressão no fluido.
- $\mu \nabla^2 \mathbf{u}$: Termo viscoso (difusão de momento), correspondente ao atrito interno do fluido — o único efeito isolado e simulado no código de diferenças finitas deste trabalho.
- $\mathbf{f}$: Forças externas volumétricas, como a gravidade ou forças eletromagnéticas.

Equação da Continuidade: além do momento, as simulações de fluidos exigem a conservação da massa. Para fluidos incompressíveis, ela se resume a $\nabla \cdot \mathbf{u} = 0$ — a divergência do campo de velocidades deve ser nula em qualquer ponto (o que entra em um volume deve sair imediatamente).

Aplicações e Desafios Matemáticos: as aplicações cobrem a previsão do tempo, aerodinâmica de aeronaves, projeto de turbinas, hidrodinâmica naval e computação gráfica para simulação de fumaça e água. Do ponto de vista teórico, as equações de Navier-Stokes constituem um dos sete Problemas do Prêmio do Milênio do Clay Mathematics Institute: até hoje não se provou formalmente se, dadas condições iniciais suaves em 3D, sempre existirão soluções suaves para todo o tempo futuro, ou se o sistema pode desenvolver singularidades.

2.2. Ambiente de execução (NPAD) e script SLURM

Para testar foi usado o script abaixo, submetido ao NPAD, baseado na documentação do NPAD e do PaScal Suite:

```
#!/bin/bash
#SBATCH --job-name=tarefa_12
#SBATCH --time=0-0:30
#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH --cpus-per-task=32
#SBATCH --output=%x_%j.out

cd ~/Tarefa_12
source ~/pascal-releases-master/env.sh

gcc -O3 -march=native -fopenmp navier_stokes_p.c -lmpascalops -o navier_stokes_p_v1 -lm

pascalanalyzer ./navier_stokes_p_v1 -t man -g -r 5 --idtm 1 -c 1,2,4,8,16,32 -i 10,20,40,80,160,320 -v INFO -o
navier_stokes_v1.json
```

Script 1 — Submissão SLURM no nó amd-512 (32 núcleos, exclusivo) com instrumentação via PaScal Analyzer

Para o teste v1 é utilizada a flag `-O3 -march=native`, para verificar se a otimização do compilador por si só já traz ganhos de eficiência/escalabilidade para um problema deste tipo (stencil 3D memory-bound).

Tabela de Comparação: Com vs. Sem Flags de Compilação

Característica	Sem Flags (-O0)	Com Flags (-O3 -march=native)
Velocidade de Execução	Mais lenta (foco em depuração)	Máxima velocidade possível
Tempo de Compilação	Quase instantâneo	Mais demorado (compilador analisa mais)

Portabilidade	Roda em qualquer PC compatível	Apenas na sua máquina (ou CPUs iguais)
Tamanho do Arquivo	Menor / Padrão	Geralmente maior (devido às otimizações)

3. Código Fonte

Para a versão sequencial-paralela base foi usado o código abaixo, gerado com apoio de uma IA (conforme recomendado pelo professor em aula, já que o foco da tarefa é a avaliação de escalabilidade, não a modelagem física de Navier-Stokes). Ao todo foram avaliadas quatro versões: v0 (paralelização apenas do laço de difusão, sem otimização de compilador), v1 (mesmo código-fonte do v0, recompilado com -O3 -march=native — por isso não há uma seção de código v1 separada), v2 (paraleliza também a função da perturbação gaussiana, mas sem instrumentá-la com pascal_start/pascal_stop) e v3 (repete a v2, agora instrumentando a gaussiana com pascal_start(2)/pascal_stop(2) para que ela também entre no cálculo de eficiência).

3.1. Código Navier Stokes — v0 (e v1, recompilado com -O3 -march=native)

```
#define _POSIX_C_SOURCE 199309L

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>
#include "pascalops.h"

#define LX 1.0
#define LY 1.0
#define LZ 1.0
#define NU 0.01

#define AMPLITUDE 2.0
#define LARGURA 0.03

#define NX_PADRAO 64
#define NY_PADRAO 64
```

```
#define NZ_PADRAO 64

#define N_STEPS_PADRAO 200

#define CSV_PADRAO "difusao3d_p_collapse2.csv"

#define MAX_PTS_PADRAO 48

#define N_SNAPS_PADRAO 5


static double agora_em_segundos(void) {

    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;

}


static inline long idx(int i, int j, int k, int NY, int NZ) {

    return ((long) i * NY + j) * NZ + k;

}


static double *aloca_campo(int NX, int NY, int NZ) {

    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));

    if (campo == NULL) {

        fprintf(stderr, "Erro: falha ao alocar memoria.\n");

        exit(EXIT_FAILURE);

    }

    return campo;

}
```

```
static void aplica_contorno(double *c, int NX, int NY, int NZ) {
```

```
    for (int i = 0; i < NX; i++)
```

```
        for (int j = 0; j < NY; j++) {
```

```
            c[idx(i, j, 0, NY, NZ)] = 0.0;
```

```
            c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;
```

```
        }
```

```
    for (int i = 0; i < NX; i++)
```

```
        for (int k = 0; k < NZ; k++) {
```

```
            c[idx(i, 0, k, NY, NZ)] = 0.0;
```

```
            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;
```

```
        }
```

```
    for (int j = 0; j < NY; j++)
```

```
        for (int k = 0; k < NZ; k++) {
```

```
            c[idx(0, j, k, NY, NZ)] = 0.0;
```

```
            c[idx(NX - 1, j, k, NY, NZ)] = 0.0;
```

```
        }
```

```
}
```

```
static void passo_difusao(const double *c, double *cn, double dt, double dx, double dy,
```

```
double dz, int NX, int NY, int NZ) {
```

```
    const double kx = NU * dt / (dx * dx);
```

```
    const double ky = NU * dt / (dy * dy);
```

```
    const double kz = NU * dt / (dz * dz);
```

```

/* Gerando a região Paralela */

pascal_start(1);

#pragma omp parallel

{

    #pragma omp for collapse(2)

    for (int i = 1; i < NX - 1; i++) {

        for (int j = 1; j < NY - 1; j++) {

            for (int k = 1; k < NZ - 1; k++) {

                long p = idx(i, j, k, NY, NZ);

                double centro = c[p];

                cn[p] = centro

                    + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])

                    + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])

                    + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);

            }

        }

    }

}

pascal_stop(1);

}

/* Gaussiana 3D no centro do dominio. */

static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz,

```

```

        int NX, int NY, int NZ) {

    for (int i = 0; i < NX; i++)

        for (int j = 0; j < NY; j++)

            for (int k = 0; k < NZ; k++) {

                double x = i * dx - 0.5 * LX;

                double y = j * dy - 0.5 * LY;

                double z = k * dz - 0.5 * LZ;

                double r2 = x * x + y * y + z * z;

                u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA *
LARGURA));

            }

    aplica_contorno(u, NX, NY, NZ);

}

/* Grava um snapshot (subamostrado com passo ex,ey,ez) no CSV. */

static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,

        double dx, double dy, double dz, long passo, double t,

        int ex, int ey, int ez) {

    long linhas = 0;

    for (int i = 0; i < NX; i += ex)

        for (int j = 0; j < NY; j += ey)

            for (int k = 0; k < NZ; k += ez) {

                fprintf(f, "%ld,%.6g,%d,%d,%d,%.5f,%.5f,%.5f,%.6g\n",

                    passo, t, i, j, k, i * dx, j * dy, k * dz,

                    u[idx(i, j, k, NY, NZ)]);

```

```

        linhas++;

    }

    return linhas;
}

int main(int argc, char **argv) {

    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;

    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;

    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;

    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;

    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;

    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;

    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {

        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MX_PTS>=2] [N_SNAPS>=2]\n", argv[0]);

        return EXIT_FAILURE;

    }

    double dx = LX / (NX - 1);

    double dy = LY / (NY - 1);

    double dz = LZ / (NZ - 1);

    double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz *
dz)));

```

```

double dt = 0.4 * limite_estabilidade;

double *u = aloca_campo(NX, NY, NZ);

double *u_novo = aloca_campo(NX, NY, NZ);

long total_pontos = (long) NX * NY * NZ;

aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);

FILE *f = fopen(arq_csv, "w");

if (f == NULL) {

    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);

    return EXIT_FAILURE;

}

fprintf(f, "passo,t,i,j,k,x,y,z,u\n");

int ex = (NX + max_pts - 1) / max_pts;

int ey = (NY + max_pts - 1) / max_pts;

int ez = (NZ + max_pts - 1) / max_pts;

fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
Paralelo | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);

double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */

long linhas = 0;

int s = 0;

```

```

for (long passo = 0; passo <= N_STEPS; passo++) {

    /* snapshots igualmente espacados entre o passo 0 e o final */

    if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {

        linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey,
ez);

        while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;

    }

    if (passo == N_STEPS) break;


    double inicio = agora_em_segundos();

    passo_difusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);

    double *tmp = u; u = u_novo; u_novo = tmp;    /* troca de ponteiros, sem memcpy */

    tempo_calculo += agora_em_segundos() - inicio;

}

fclose(f);


double soma = 0.0;

for (long p = 0; p < total_pontos; p++) soma += u[p];


fprintf(stderr, "(soma de u) = %.10f\n", soma);

fprintf(stderr, "[csv] %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n",
arq_csv, s, linhas, ex, ey, ez);

printf("Tempo: %.6f s\n", tempo_calculo);

```

```
free(u);  
  
free(u_novo);  
  
return 0;  
  
}
```

Código 1 — navier_stokes_p_v0.c (base da v0 e da v1; v1 usa este mesmo arquivo compilado com -O3 -march=native)

3.2. Código Navier Stokes — v2

Diferença em relação à v0/v1: a função aplica_perturbacao_gaussiana passa a ser paralelizada com `#pragma omp parallel for collapse(3)`, mas ainda sem marcação `pascal_start/pascal_stop` — ou seja, essa região continua fora do escopo medido pelo PaScal Analyzer.

```
#define _POSIX_C_SOURCE 199309L

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <omp.h>
#include "pascalops.h"

#define LX 1.0
#define LY 1.0
#define LZ 1.0
#define NU 0.01

#define AMPLITUDE 2.0
#define LARGURA 0.03

#define NX_PADRAO 64
#define NY_PADRAO 64
#define NZ_PADRAO 64
#define N_STEPS_PADRAO 200
#define CSV_PADRAO "difusao3d_p_collapse2.csv"
#define MAX_PTS_PADRAO 48
```

```
#define N_SNAPS_PADRAO 5

static double agora_em_segundos(void) {

    struct timespec ts;

    clock_gettime(CLOCK_MONOTONIC, &ts);

    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;

}

static inline long idx(int i, int j, int k, int NY, int NZ) {

    return ((long) i * NY + j) * NZ + k;

}

static double *aloca_campo(int NX, int NY, int NZ) {

    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));

    if (campo == NULL) {

        fprintf(stderr, "Erro: falha ao alocar memoria.\n");

        exit(EXIT_FAILURE);

    }

    return campo;

}

static void aplica_contorno(double *c, int NX, int NY, int NZ) {

    for (int i = 0; i < NX; i++)

        for (int j = 0; j < NY; j++) {

            c[idx(i, j, 0, NY, NZ)] = 0.0;

        }

}
```

```

        c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;
    }

    for (int i = 0; i < NX; i++)
        for (int k = 0; k < NZ; k++) {
            c[idx(i, 0, k, NY, NZ)] = 0.0;
            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;
        }

    for (int j = 0; j < NY; j++)
        for (int k = 0; k < NZ; k++) {
            c[idx(0, j, k, NY, NZ)] = 0.0;
            c[idx(NX - 1, j, k, NY, NZ)] = 0.0;
        }
}

```

```

static void passo_difusao(const double *c, double *cn, double dt, double dx, double dy,
double dz, int NX, int NY, int NZ) {

    const double kx = NU * dt / (dx * dx);
    const double ky = NU * dt / (dy * dy);
    const double kz = NU * dt / (dz * dz);

    /* Gerando a região Paralela */

    pascal_start(1);

    #pragma omp parallel
    {

```

```

#pragma omp for collapse(2)

for (int i = 1; i < NX - 1; i++) {

    for (int j = 1; j < NY - 1; j++) {

        for (int k = 1; k < NZ - 1; k++) {

            long p = idx(i, j, k, NY, NZ);

            double centro = c[p];

            cn[p] = centro

                + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])

                + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])

                + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);

        }

    }

}

pascal_stop(1);

}

/* Gaussiana 3D no centro do dominio. */

static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz, int
NX, int NY, int NZ) {

    #pragma omp parallel for collapse(3)

    for (int i = 0; i < NX; i++)

```

```

for (int j = 0; j < NY; j++)

    for (int k = 0; k < NZ; k++) {

        double x = i * dx - 0.5 * LX;

        double y = j * dy - 0.5 * LY;

        double z = k * dz - 0.5 * LZ;

        double r2 = x * x + y * y + z * z;

        u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA *
LARGURA));

    }

    aplica_contorno(u, NX, NY, NZ);

}

/* Grava um snapshot (subamostrado com passo ex,ey,ez) no CSV. */
static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,

        double dx, double dy, double dz, long passo, double t,

        int ex, int ey, int ez) {

    long linhas = 0;

    for (int i = 0; i < NX; i += ex)

        for (int j = 0; j < NY; j += ey)

            for (int k = 0; k < NZ; k += ez) {

                fprintf(f, "%ld,%.6g,%d,%d,%d,%.5f,%.5f,%.5f,%.6g\n",

                    passo, t, i, j, k, i * dx, j * dy, k * dz,

                    u[idx(i, j, k, NY, NZ)]);

                linhas++;

            }

}

```

```

return linhas;
}

int main(int argc, char **argv) {

    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;

    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;

    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;

    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;

    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;

    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;

    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {

        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MX_PTS>=2] [N_SNAPS>=2]\n", argv[0]);

        return EXIT_FAILURE;

    }

    double dx = LX / (NX - 1);

    double dy = LY / (NY - 1);

    double dz = LZ / (NZ - 1);

    double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz *
dz)));

    double dt = 0.4 * limite_estabilidade;

```

```

double *u = aloca_campo(NX, NY, NZ);

double *u_novo = aloca_campo(NX, NY, NZ);

long total_pontos = (long) NX * NY * NZ;

aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);

FILE *f = fopen(arq_csv, "w");

if (f == NULL) {

    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);

    return EXIT_FAILURE;

}

fprintf(f, "passo,t,i,j,k,x,y,z,u\n");


int ex = (NX + max_pts - 1) / max_pts;

int ey = (NY + max_pts - 1) / max_pts;

int ez = (NZ + max_pts - 1) / max_pts;


fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
Paralela | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);


double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */

long linhas = 0;

int s = 0;


for (long passo = 0; passo <= N_STEPS; passo++) {

```

```

/* snapshots igualmente espacados entre o passo 0 e o final */

if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {

    linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey,
ez);

    while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;

}

if (passo == N_STEPS) break;


double inicio = agora_em_segundos();

passo_difusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);

double *tmp = u; u = u_novo; u_novo = tmp;    /* troca de ponteiros, sem memcpy */

tempo_calculo += agora_em_segundos() - inicio;

}

fclose(f);


double soma = 0.0;

for (long p = 0; p < total_pontos; p++) soma += u[p];


fprintf(stderr, "(soma de u) = %.10f\n", soma);

fprintf(stderr, "[csv] %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n",
arq_csv, s, linhas, ex, ey, ez);

printf("Tempo: %.6f s\n", tempo_calculo);


free(u);

free(u_novo);

```

```
return 0;  
}
```

Código 2 — navier_stokes_p_v2.c

3.3. Código Navier Stokes — v3

Diferença em relação à v2: adiciona-se pascal_start(2)/pascal_stop(2) em torno da gaussiana, incluindo-a formalmente na região analisada pelo PaScal (agora como região 0.2, filha da região 0).

```
#define _POSIX_C_SOURCE 199309L

#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <time.h>

#include <omp.h>

#include "pascalops.h"


#define LX 1.0

#define LY 1.0

#define LZ 1.0

#define NU 0.01


#define AMPLITUDE 2.0

#define LARGURA 0.03


#define NX_PADRAO 64

#define NY_PADRAO 64

#define NZ_PADRAO 64

#define N_STEPS_PADRAO 200

#define CSV_PADRAO "difusao3d_p_collapse2.csv"

#define MAX_PTS_PADRAO 48
```

```
#define N_SNAPS_PADRAO 5

static double agora_em_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double) ts.tv_sec + (double) ts.tv_nsec / 1e9;
}

static inline long idx(int i, int j, int k, int NY, int NZ) {
    return ((long) i * NY + j) * NZ + k;
}

static double *aloca_campo(int NX, int NY, int NZ) {
    double *campo = (double *) calloc((size_t) NX * NY * NZ, sizeof(double));
    if (campo == NULL) {
        fprintf(stderr, "Erro: falha ao alocar memoria.\n");
        exit(EXIT_FAILURE);
    }
    return campo;
}

static void aplica_contorno(double *c, int NX, int NY, int NZ) {
    for (int i = 0; i < NX; i++)
        for (int j = 0; j < NY; j++) {
            c[idx(i, j, 0, NY, NZ)] = 0.0;
        }
}
```

```

        c[idx(i, j, NZ - 1, NY, NZ)] = 0.0;

    }

    for (int i = 0; i < NX; i++)

        for (int k = 0; k < NZ; k++) {

            c[idx(i, 0, k, NY, NZ)] = 0.0;

            c[idx(i, NY - 1, k, NY, NZ)] = 0.0;

        }

    for (int j = 0; j < NY; j++)

        for (int k = 0; k < NZ; k++) {

            c[idx(0, j, k, NY, NZ)] = 0.0;

            c[idx(NX - 1, j, k, NY, NZ)] = 0.0;

        }

}

```

```

static void passo_difusao(const double *c, double *cn, double dt, double dx, double dy,
double dz, int NX, int NY, int NZ) {

    const double kx = NU * dt / (dx * dx);

    const double ky = NU * dt / (dy * dy);

    const double kz = NU * dt / (dz * dz);

    /* Gerando a região Paralela */

    pascal_start(1);

    #pragma omp parallel

    {

```

```

#pragma omp for collapse(2)

for (int i = 1; i < NX - 1; i++) {

    for (int j = 1; j < NY - 1; j++) {

        for (int k = 1; k < NZ - 1; k++) {

            long p = idx(i, j, k, NY, NZ);

            double centro = c[p];

            cn[p] = centro

                + kx * (c[idx(i + 1, j, k, NY, NZ)] - 2.0 * centro + c[idx(i - 1, j, k, NY, NZ)])

                + ky * (c[idx(i, j + 1, k, NY, NZ)] - 2.0 * centro + c[idx(i, j - 1, k, NY, NZ)])

                + kz * (c[idx(i, j, k + 1, NY, NZ)] - 2.0 * centro + c[idx(i, j, k - 1, NY, NZ)]);

        }

    }

}

pascal_stop(1);

}

/* Gaussiana 3D no centro do dominio. */

static void aplica_perturbacao_gaussiana(double *u, double dx, double dy, double dz, int
NX, int NY, int NZ) {

    pascal_start(2);

    #pragma omp parallel for collapse(3)

    for (int i = 0; i < NX; i++)

```

```

for (int j = 0; j < NY; j++)

    for (int k = 0; k < NZ; k++) {

        double x = i * dx - 0.5 * LX;

        double y = j * dy - 0.5 * LY;

        double z = k * dz - 0.5 * LZ;

        double r2 = x * x + y * y + z * z;

        u[idx(i, j, k, NY, NZ)] += AMPLITUDE * exp(-r2 / (2.0 * LARGURA *
LARGURA));

    }

pascal_stop(2);

aplica_contorno(u, NX, NY, NZ);

}

/* Grava um snapshot (subamostrado com passo ex,ey,ez) no CSV. */
static long grava_snapshot(FILE *f, const double *u, int NX, int NY, int NZ,

        double dx, double dy, double dz, long passo, double t,

        int ex, int ey, int ez) {

long linhas = 0;

for (int i = 0; i < NX; i += ex)

    for (int j = 0; j < NY; j += ey)

        for (int k = 0; k < NZ; k += ez) {

            fprintf(f, "%ld,%.6g,%d,%d,%d,%.5f,%.5f,%.5f,%.6g\n",

                passo, t, i, j, k, i * dx, j * dy, k * dz,

                u[idx(i, j, k, NY, NZ)]);

            linhas++;

```

```

    }

    return linhas;
}

int main(int argc, char **argv) {

    int NX = (argc > 1) ? atoi(argv[1]) : NX_PADRAO;

    int NY = (argc > 2) ? atoi(argv[2]) : NY_PADRAO;

    int NZ = (argc > 3) ? atoi(argv[3]) : NZ_PADRAO;

    long N_STEPS = (argc > 4) ? atol(argv[4]) : N_STEPS_PADRAO;

    const char *arq_csv = (argc > 5) ? argv[5] : CSV_PADRAO;

    int max_pts = (argc > 6) ? atoi(argv[6]) : MAX_PTS_PADRAO;

    int n_snaps = (argc > 7) ? atoi(argv[7]) : N_SNAPS_PADRAO;

    if (NX < 3 || NY < 3 || NZ < 3 || N_STEPS < 1 || max_pts < 2 || n_snaps < 2) {

        fprintf(stderr, "Uso: %s [NX>=3] [NY>=3] [NZ>=3] [N_STEPS>=1] [saida.csv]
[MX_PTS>=2] [N_SNAPS>=2]\n", argv[0]);

        return EXIT_FAILURE;

    }

    double dx = LX / (NX - 1);

    double dy = LY / (NY - 1);

    double dz = LZ / (NZ - 1);

    double limite_estabilidade = 0.5 / (NU * (1.0 / (dx * dx) + 1.0 / (dy * dy) + 1.0 / (dz *
dz)));

    double dt = 0.4 * limite_estabilidade;

```

```
double *u = aloca_campo(NX, NY, NZ);

double *u_novo = aloca_campo(NX, NY, NZ);

long total_pontos = (long) NX * NY * NZ;


aplica_perturbacao_gaussiana(u, dx, dy, dz, NX, NY, NZ);


FILE *f = fopen(arq_csv, "w");

if (f == NULL) {

    fprintf(stderr, "Erro: nao foi possivel abrir '%s'.\n", arq_csv);

    return EXIT_FAILURE;

}

fprintf(f, "passo,t,i,j,k,x,y,z,u\n");


int ex = (NX + max_pts - 1) / max_pts;

int ey = (NY + max_pts - 1) / max_pts;

int ez = (NZ + max_pts - 1) / max_pts;


fprintf(stderr, "Malha = %d x %d x %d (%.2f milhoes de pontos) | passos = %ld |
Paralela | dt = %.3e\n", NX, NY, NZ, total_pontos / 1e6, N_STEPS, dt);


double tempo_calculo = 0.0; /* so os passos; a gravacao do CSV fica de fora */

long linhas = 0;

int s = 0;
```

```

for (long passo = 0; passo <= N_STEPS; passo++) {

    /* snapshots igualmente espaçados entre o passo 0 e o final */

    if (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) {

        linhas += grava_snapshot(f, u, NX, NY, NZ, dx, dy, dz, passo, passo * dt, ex, ey,
ez);

        while (s < n_snaps && passo == (long) s * N_STEPS / (n_snaps - 1)) s++;

    }

    if (passo == N_STEPS) break;

    double inicio = agora_em_segundos();

    passo_difusao(u, u_novo, dt, dx, dy, dz, NX, NY, NZ);

    double *tmp = u; u = u_novo; u_novo = tmp;    /* troca de ponteiros, sem memcpy */

    tempo_calculo += agora_em_segundos() - inicio;

}

fclose(f);

double soma = 0.0;

for (long p = 0; p < total_pontos; p++) soma += u[p];

fprintf(stderr, "(soma de u) = %.10f\n", soma);

fprintf(stderr, "[csv] %s: %d snapshots, %ld linhas (amostragem %dx%dx%d)\n",
arq_csv, s, linhas, ex, ey, ez);

printf("Tempo: %.6f s\n", tempo_calculo);

free(u);

```

```
free(u_novo);  
  
return 0;  
  
}
```

Código 3 — navier_stokes_p_v3.c

4. Resultado

Cada versão foi executada no nó amd-512 do NPAD (32 núcleos, execução exclusiva) via PaScaL Analyzer, variando o número de núcleos (1, 2, 4, 8, 16, 32) e o tamanho da malha (NX = 10, 20, 40, 80, 160, 320), na mesma razão de crescimento (dobrando a cada passo), conforme recomenda a documentação do PaScaL para permitir a leitura de escalabilidade fraca na diagonal dos mapas de calor. Os sete painéis gerados pelo PaScaL Viewer para cada versão são: Efficiency, Performance, Scalability, Speedup, Speedup Loss, Strong Scalability e Weak Scalability.

4.1. Versão v0 — apenas collapse(2) na difusão, sem otimização de compilador

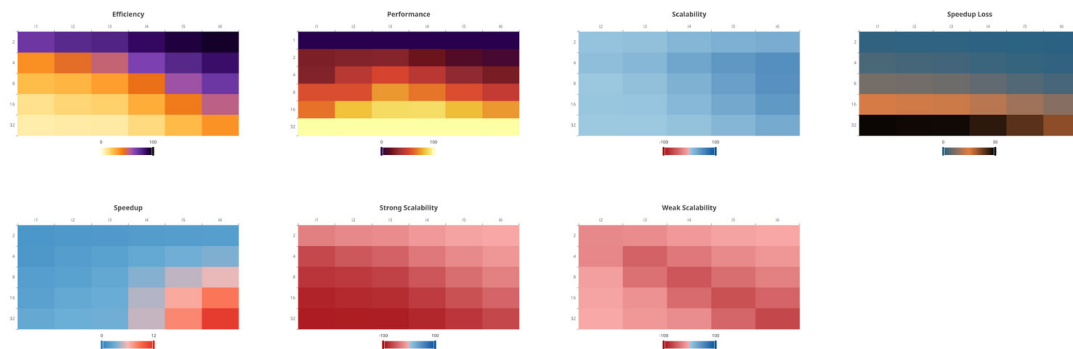


Figura 2 — Painéis de escalabilidade da v0 (navier_stokes_v0.json)

O painel de Efficiency mostra o padrão clássico esperado pela Lei de Amdahl: eficiência próxima de 100% no canto de poucos núcleos e malha grande, caindo para próximo de 0% com 32 núcleos na malha menor ($i1 = 10^3$ pontos) — nessa configuração o overhead de criar e sincronizar 32 threads passa a dominar sobre o trabalho real por thread. O painel de Speedup revela um teto de aproximadamente 12× de aceleração máxima observada. O painel de Speedup Loss (perda de speedup em relação ao ideal) confirma que essa perda cresce de forma consistente com o número de núcleos, mais acentuadamente nas malhas pequenas — evidência direta de um gargalo dominado por overhead de paralelização (fração serial fixa por passo de tempo), não por falta de trabalho paralelizável.

4.2. Versão v1 — mesmo código-fonte, compilado com -O3 -march=native

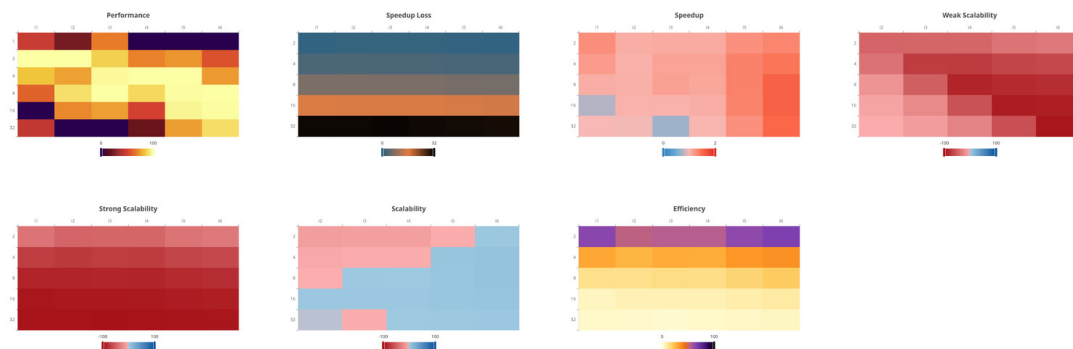


Figura 3 — Painéis de escalabilidade da v1 (navier_stokes_v1.json)

Este é o resultado mais revelador do trabalho: o teto do painel de Speedup cai de $\sim 12\times$ (v0) para apenas $\sim 2\times$ em v1. A otimização de compilador tornou o laço de difusão (região paralela) tão mais rápido que a fração serial fixa do programa (soma final, gravação do CSV e overhead de criação de threads a cada um dos 200 passos de tempo) passou a dominar o tempo total — exatamente o comportamento previsto pela Lei de Amdahl: acelerar a parte paralela sem reduzir a parte serial fixa reduz o speedup máximo alcançável, mesmo que o tempo absoluto de execução tenha diminuído. Em outras palavras, -O3 -march=native deixou o programa mais rápido, mas não mais escalável — e, sob a ótica de escalabilidade, até revelou um teto mais severo do que o aparente na v0.

4.3. Versão v2 — gaussiana paralelizada, sem instrumentação PaScal

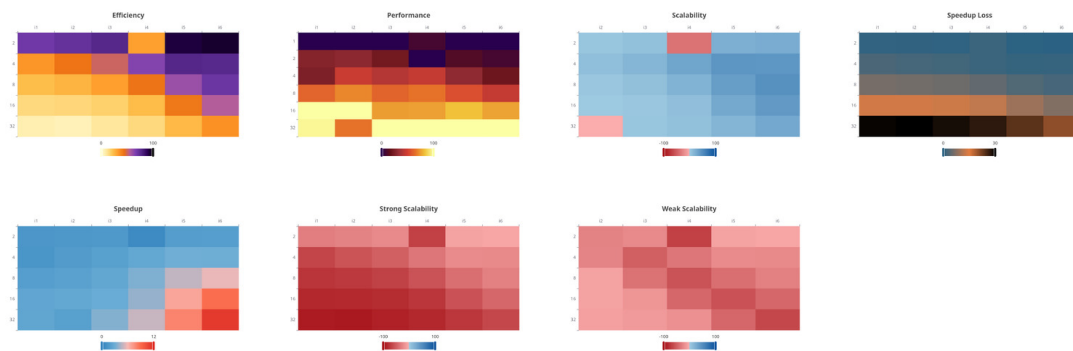


Figura 4 — Painéis de escalabilidade da v2 (navier_stokes_v2.json)

O teto de Speedup volta a $\sim 12\times$, no mesmo patamar da v0, e os demais painéis (Efficiency, Scalability, Strong/Weak Scalability) reproduzem visualmente o padrão da v0. Isso é coerente com o fato de que a região medida pelo PaScal (pascal_start(1)/pascal_stop(1)) continua sendo exclusivamente o laço de difusão — a paralelização da gaussiana nesta versão não está instrumentada e, portanto, é invisível para a métrica de escalabilidade, mesmo que ela ajude (ou não) o tempo total de execução por fora da medição. É importante registrar, para fins de defesa, se a v2 foi ou não recompilada com -O3 -march=native: a coincidência do teto de speedup com o da v0 (e não com o da v1) sugere que a v2 pode ter sido testada sem essa flag de otimização — vale confirmar isso antes da apresentação, pois muda a leitura do porquê do teto ter retornado a $12\times$.

4.4. Versão v3 — gaussiana instrumentada com pascal_start(2)/pascal_stop(2)

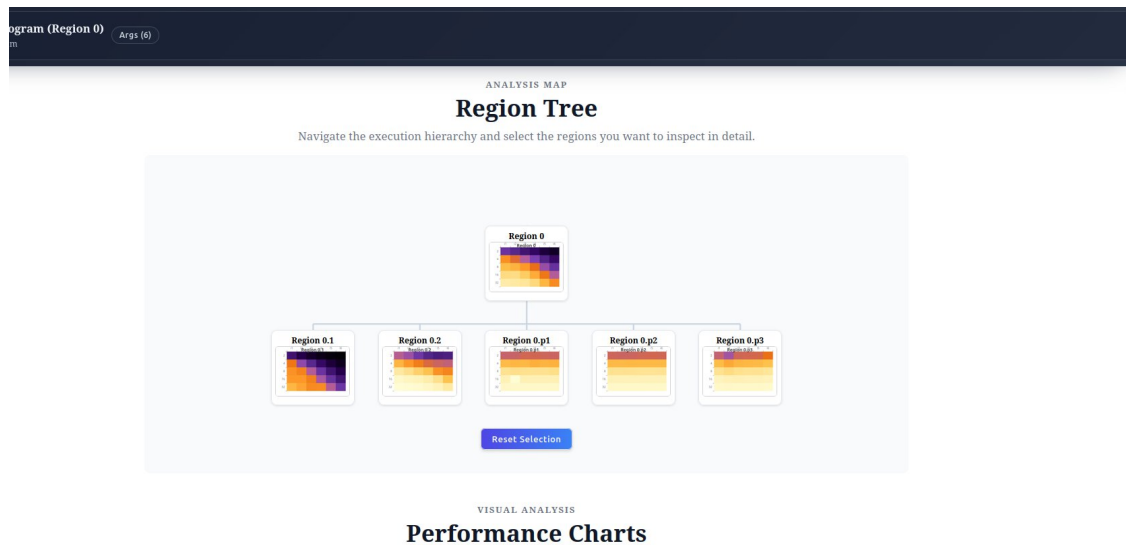


Figura 5 — Árvore de regiões da v3: Região 0 (programa inteiro) com filhas 0.1 (difusão), 0.2 (gaussiana), 0.p1/0.p2/0.p3 (trechos não instrumentados/seriais)

Com a gaussiana agora incluída como região 0.2, a análise por região do PaScal mostra que essa região representa apenas entre 0% e 4% do tempo total de execução em todas as configurações testadas, contra 2%–46% da região 0.1 (difusão) e 1%–49% somados nas regiões de gap 0.p1/0.p2/0.p3 (código serial: soma final, gravação de snapshot e overhead de criação de threads). Pela própria Lei de Amdahl aplicada a este dado: mesmo que a gaussiana fosse paralelizada com eficiência perfeita, o ganho máximo possível no tempo total do programa seria de apenas ~4% — o que confirma, com dado real e não apenas hipótese, que otimizar a gaussiana nunca seria o caminho para resolver o verdadeiro gargalo de escalabilidade do código.

5. Conclusão

Este trabalho colocou em prática, sobre um código real de simulação (difusão viscosa 3D, um subconjunto da equação de Navier-Stokes), os conceitos teóricos de desempenho e escalabilidade apresentados em aula: Desempenho = $1/\text{Tempo}$; Eficiência = $\text{Trabalho}/(\text{Tempo} \times \text{Recursos})$; $\text{Speedup}(p) = T(1)/T(p)$ (modelo de Amdahl, problema de tamanho fixo); e $\text{Eficiência}(p) = S(p)/p$ (modelo de Gustafson, problema crescendo com os recursos).

5.1. Escalabilidade (geral)

Segundo a classificação apresentada em aula — programas escaláveis são aqueles capazes de compensar a perda de eficiência aumentando o tamanho do problema junto com os recursos — o código avaliado é escalável, mas de forma limitada. Em nenhuma das quatro versões o programa alcança e mantém 100% de eficiência ao mesmo tempo em que aumenta o número de núcleos: há sempre uma fração de overhead (criação de threads a cada passo de tempo, mais o código estritamente serial de soma e I/O) que limita o ganho, ficando mais evidente quanto mais rápida fica a parte paralela do código ($v1$ em relação à $v0$).

5.2. Escalabilidade Forte

Lendo cada coluna dos mapas de calor de cima para baixo (tamanho de malha fixo, número de núcleos crescendo), a eficiência degrada de forma consistente em todas as versões — mais acentuadamente nas malhas pequenas ($i1 = 10^3$ pontos), onde o overhead de sincronização de 32 threads supera o trabalho útil por thread. O painel de Strong Scalability confirma isso numericamente: os valores tendem para o vermelho (perda) conforme os núcleos aumentam, praticamente independente do tamanho do problema. Isso classifica o código como fracamente escalável na melhor das versões ($v0/v2$), e ainda mais limitado na $v1$, onde o teto de speedup de apenas $\sim 2\times$ mostra que o código não é fortemente escalável — não sustenta eficiência ao simplesmente adicionar núcleos sem crescer a malha.

5.3. Escalabilidade Fraca

Lendo a diagonal dos mapas (núcleos e malha crescendo juntos, na mesma proporção usada nos parâmetros `-c` e `-i` do PaScal Analyzer), a eficiência se mantém mais estável do que na leitura de escalabilidade forte, mas ainda apresenta degradação visível nas configurações de maior número de núcleos, especialmente na $v1$. Isso indica que o código é parcialmente fracamente escalável: crescer o problema junto com os recursos ajuda a esconder o overhead fixo por passo de tempo, mas não o elimina — o custo de recriar a região paralela 200 vezes (uma por passo de tempo) cresce em proporção ao número de núcleos, então mesmo mantendo o trabalho por núcleo constante, o overhead absoluto por passo de tempo aumenta com mais threads.

5.4. Gargalo identificado

O verdadeiro limitador de escalabilidade deste código não está em qual função é paralelizada (a gaussiana, provado pela região 0.2 na $v3$, responde por no máximo 4% do tempo total), e sim na estrutura do laço principal: a diretiva `#pragma omp parallel` dentro de `passo_difusao` é executada — e portanto recria threads — 200 vezes (uma por passo de tempo), e o código estritamente serial que sobra fora dela (soma de verificação final, gravação de snapshots) tem custo fixo que não diminui com

mais núcleos. Pela Lei de Amdahl, essa fração serial fixa é o que impõe um teto de speedup — teto que fica mais evidente (e mais baixo, $\sim 2\times$) exatamente quando o código paralelo é otimizado por compilador (v1), pois a parte que ficou mais rápida foi a parte paralela, não a serial.